

pSearchGrid and pSpoofNode

Search-grid and spoof-node utilities.

Included MIT MOOS-IvP PDF sources:

- app_psearchgrid.pdf
- app_pspoofnode.pdf

pSearchGrid: Using a 2D Grid Model for Track History

June 2018

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview	1
2	Using pSearchGrid	2
2.1	Basic Configuration of Grid Cells	2
2.2	Cell Variables	3
2.3	Serializing and De-serializing the Grid Structure	3
2.4	Resetting the Grid	4
2.5	Viewing Grids in pMarineViewer	4
2.6	Examples	4
3	Configuration Parameters of pSearchGrid	4
4	Publications and Subscriptions for pSearchGrid	5
4.1	Variables Published by pSearchGrid	5
4.2	Variables Subscribed for by pSearchGrid	5
4.3	Command Line Usage of pSearchGrid	6

1 Overview

The `pSearchGrid` application is a module for storing a history of vehicle positions in a 2D grid defined over a region of operation. This module may have utility as-is, to help guide a vehicle to complete or uniform coverage of a given area, but also exists as an example of how to use and visualize the `XYConvexGrid` data structure. This data structure may be used in similar modules to store a wide variety of user specified data for later use by other modules or simply for visualization. Here the structure is used in a MOOS application, but it could also be used within a behavior. The `pSearchGrid` module begins with a user-defined grid, defined in the MOOS configuration file. As a vehicle moves and generates node reports, `pSearchGrid` simply notes the vehicle's current position increments the cell containing vehicle's current position. After time, the grid shows a cumulative history of the most commonly traveled positions in the local operating area.

The `pSearchGrid` module is an optional part of the Charlie example mission discussed later in this section. When running and grid viewing is enabled in `pMarineViewer`, the viewer may show something similar to Figure 1.

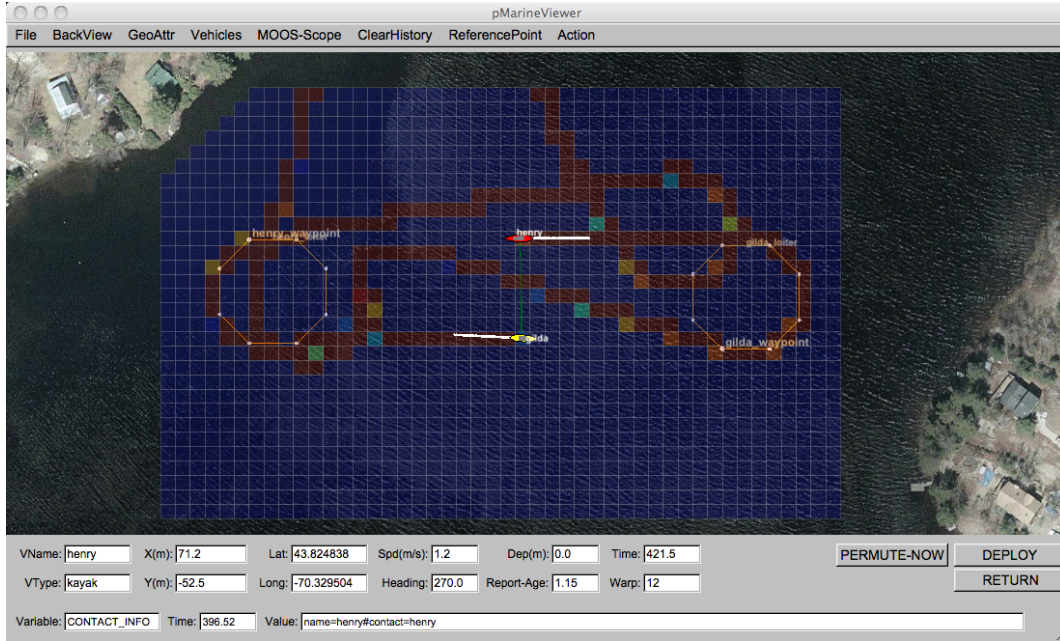


Figure 1: **An Example pSearchGrid Scenario:** A grid is configured around the operation area of the Berta example mission. Higher values in a given grid cell represents a longer noted time of any vehicle passing through that cell.

2 Using pSearchGrid

The present configuration of pSearchGrid will store values in its grid cells proportional to the time a vehicle is noted to be within a grid cell. One may use this application as written or regard this as a template for writing a new application that stores some other value with each grid cell, such as bathymetry data, water velocity data, or likelihood of there being an object of interest in the region of the cell for further investigation. The below discusses the general usage of managing the grid data within a MOOS application.

2.1 Basic Configuration of Grid Cells

Basic grid configuration consists of specifying (a) a convex polygon, and (b) the size of the grid squares. From this the construction of a grid proceeds by calculating a bounding rectangle containing the polygon, generating a set of squares covering the rectangle, and then removing the squares not intersecting the original polygon. The result is a non-rectilinear grid as shown in Figure 1. The basic configuration of grid cells is done with the `grid_config` parameter specifying the points and the cell size as the following example:

```
grid_config = pts={-50,-40:-10,0:180,0:180,-150:-50,-150}, cell_size=5
```

2.2 Cell Variables

The grid is primary used for associating information with each grid cell. In `pSearchGrid`, the grid is configured to store a single numerical value (a C++ double) with each cell. The grid structure may be configured in another application to store multiple numerical values with each cell. In `pSearchGrid`, the cell variables are declared upon startup in the MOOS configuration block, similar to:

```
grid_config = cell_vars=x:0y:100
grid_config = cell_min=x:0
grid_config = cell_min=x:100
```

This configuration associates two cell variables, x and y , with each cell. The cell variable x is initialized to 0 for each cell, and the cell variable y is initialized to 100. The first variable has a minimum and maximum constraint of 0 and 100, and the second variable is unconstrained. The above configuration could also have been made on one line, separating each with a comma.

2.3 Serializing and De-serializing the Grid Structure

The grid structure maintained by `pSearchGrid` is periodically published to the MOOSDB for consumption by other applications, or conceivably by a behavior with the IvP Helm. The structure may be serialized into a string by calling the `get_spec()` method on an instance of the `XYConvexGrid` class. Serializing a grid instance and posting it to the MOOSDB may look something like:

```
#include "XYConvexGrid.h"
....
XYConvexGrid mygrid;
....
string str = mygrid.get_spec();
m_Comms.Notify(VIEW_GRID, str);
```

Likewise a string representation of the grid may be de-serialized to a grid instance by calling a function provided in the same library where the grid is defined. De-serializing a string read from the MOOSDB into a local grid instance may look something like:

```
#include "XYFormatUtilsConvexGrid.h"
#include "XYConvexGrid.h"
....
string grid_string_spec;
....
XYConvexGrid mygrid = string2ConvexGrid(grid_string_spec);
```

The same function used to de-serialize a string is used by `pSearchGrid` to configure the initial grid. In other words, the components from the various GRID configuration parameters are concatenated into a single comma-separated string and passed to the de-serialization function, `string2ConvexGrid`, to form the initial instance used by `pSearchGrid`.

2.4 Resetting the Grid

The grid may be reset at any point in the operation when `pSearchGrid` receives mail on the variable `PSG_GRID_RESET`. If this variable's string value is the empty string, it will reset all cell variables for all cell elements to their given initial values. If the string value is non-empty, it will interpret this as an attempt to reset the values for a named cell variable. Thus `PSG_GRID_RESET="x"` will reset the x cell variable and no other cell variables. If "x" is not a cell variable, no action will be taken.

2.5 Viewing Grids in pMarineViewer

The `pMarineViewer` will display grids by subscribing for postings to the variable `VIEW_GRID`. As with other viewable objects such as polygons, points, etc., the viewer keeps a local cache of grid instances, one for each named grid, based on the grid label. For example if two successive grids are received with different labels, the viewer will store and render both of them. If they have the same label, the second will replace the first and the viewer will just render the one.

The rendering of grids may be toggled on/off by hitting the 'g' key, and may be made more transparent with the CTRL-g key, and less transparent with the ALT-g key. The color map used for the grid is taken from the typical MATLAB color map; red is the highest value, blue is the lowest.

2.6 Examples

Example usage of `pSearchGrid` may be found in both the Charlie and Berta example missions distributed with the moos-ivp tree. For example, in the Charlie example mission, edit `charlie.moos` and uncomment the line beginning with `Run = pSearchGrid`. Likewise for the Berta mission and the file `meta.shoreside.moos`.

3 Configuration Parameters of pSearchGrid

The following parameters are defined for `pSearchGrid`. A more detailed description is provided in other parts of this section. Parameters having default values indicate so.

Listing 3.1: Configuration Parameters for pSearchGrid.

`grid_config:` A portion or all of a grid configuration description. See Listing 2.

An Example MOOS Configuration Block

An example `pSearchGrid` configuration block is given in Listing 2 below, and may also be seen from the command line invocation of:

```
$ pSearchGrid --example or -e
```

Listing 3.2: Example configuration of the pSearchGrid application.

```
1 =====
2 pSearchGrid Example MOOS Configuration
3 =====
```

```

4
5 ProcessConfig = pSearchGrid
6 {
7     AppTick    = 4
8     CommsTick  = 4
9
10    GRID_CONFIG = pts={-50,-40: -10,0: 180,0: 180,-150: -50,-150}
11    GRID_CONFIG = cell_size=5
12    GRID_CONFIG = cell_vars=x:0:y:0:z:0
13    GRID_CONFIG = cell_min=x:0
14    GRID_CONFIG = cell_max=x:10
15    GRID_CONFIG = cell_min=y:0
16    GRID_CONFIG = cell_max=y:1000
17 }

```

4 Publications and Subscriptions for pSearchGrid

The interface for `pSearchGrid`, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ pSearchGrid --interface or -i
```

4.1 Variables Published by pSearchGrid

The `pSearchGrid` application publishes the following variables:

- **APPCAST**: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility.
- **VIEW_GRID**: A full description of a grid format and contents.

A typical string may be:

```
VIEW_GRID = pts={-50,-40:-10,0:100,0:100,-100:50,-150:-50,-100},cell_size=5,
           cell_vars=x:0:y:0:z:0,cell_min=x:0,cell_max=x:50,cell=211:x:50,
           cell=212:x:50,cell=237:x:50,cell=238:x:50,label=psg
```

4.2 Variables Subscribed for by pSearchGrid

The `pSearchGrid` application subscribes to the following MOOS variables:

- **APPCAST_REQ**: A request to generate and post a new appcast report, with reporting criteria, and expiration.
- **NODE_REPORT**: A node report for a given vehicle from `pNodeReporter`.
- **NODE_REPORT_LOCAL**: A node report for a given vehicle from `pNodeReporter`.
- **PSG_GRID_RESET**: A request to reset the grid to its original configuration.

4.3 Command Line Usage of pSearchGrid

The `pSearchGrid` application is typically launched with `pAntler`, along with a group of other modules. However, it may be launched separately from the command line. The command line options may be shown by typing "`pSearchGrid --help`":

Listing 4.3: Command line usage for the `pSearchGrid` tool.

```
1 Usage: pSearchGrid file.moos [OPTIONS]
2
3 Options:
4   --alias=<ProcessName>
5       Launch pSearchGrid with the given process
6       name rather than pSearchGrid.
7   --example, -e
8       Display example MOOS configuration block
9   --help, -h
10      Display this help message.
11   --version, -v
12      Display the release version of pSearchGrid.
13 Note: If argv[2] is not of one of the above formats
14      this will be interpreted as a run alias. This
15      is to support pAntler launching conventions.
```

pSpoofNode: Spoofing Node Reports for Non-Existing Vehicles

January 2025

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/appdocs/app_pspoofnode`

1 Overview	1
2 Configuration Parameters for pSpoofNode	2
3 Publications and Subscriptions for pSpoofNode	3
3.1 Variables Published by pSpoofNode	3
3.2 Variables Subscribed for by pSpoofNode	4
3.3 Command Line Usage of pSpoofNode	4
4 Terminal and AppCast Output	5

1 Overview

Note: This app added to MOOS-IvP after Release 24.8.

The `pSpoofNode` application is a tool for generating node report messages, as if they were being generated by vessels running `pNodeReporter`. By using this app, one or more node reports can be generated for some set initial position, heading and speed, for some duration, before they cease to be posted. In this way, applications like the helm and contact manager can be tested in a number of high contact volume edge cases to ensure correct operations, including memory management and CPU load. Typically this process is run on the shoreside MOOS community.

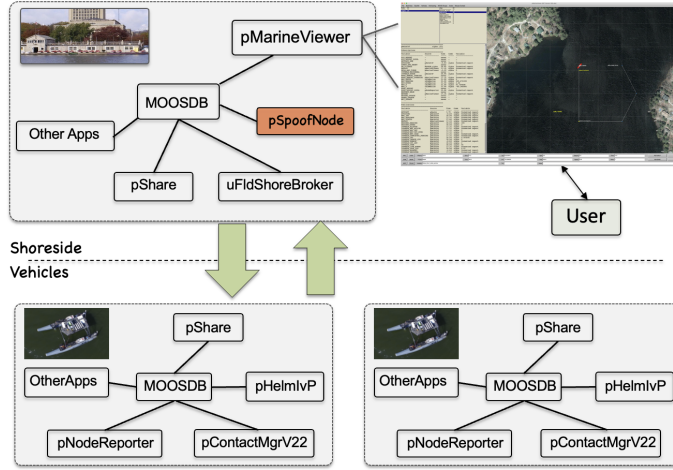


Figure 1: **Typical pSpoofNode Topology:** Node reports are generated in the shoreside community, and they are routed to all connected vehicles typically through the `uFldNodeComms` application.

2 Configuration Parameters for pSpoofNode

The `pSpoofNode` application may be configured with a configuration block within a MOOS mission file, typically with a `.moos` file suffix. The following parameters are defined for `pSpoofNode`.

Listing 2.1: Configuration Parameters for pSpoofNode.

- `default_length:` The default length of the vehicle in meters, if no vehicle length is specified in the spoof request. The default value is 5.
- `default_vtype:` The default vehicle type posted in the node report, if no vehicle type is specified in the spoof request. The default value is kayak.
- `default_group:` The default vehicle group posted in the node report, if no vehicle group is specified in the spoof request. The default value is the empty string, since the group field is optional in node reports.
- `default_vsource:` The default vehicle source posted in the node report, if no vehicle source, e.g, AIS or radar, is specified in the spoof request. The default value is the empty string, since the vsource field is optional in node reports.
- `default_color:` The default vehicle color posted in the node report, if no vehicle color is specified in the spoof request. The default value is purple.
- `default_heading:` The default vehicle heading posted in the node report, if no vehicle heading is specified in the spoof request. The default value is 45.
- `default_spd:` The default vehicle speed posted in the node report, if no vehicle speed is specified in the spoof request. The default value is 2.
- `default_duration:` The default duration, in seconds, for posting a series of node reports for a contact, if the duration is not specified in the spoof request. The default value is -1, meaning node report, if no vehicle speed is specified in the spoof there is unlimited duration.

- refresh_interval:** A minimum interval between node report publications. In seconds. The default is 1. Another way to slow down publications is with the AppTick, but this will also slow down the processing of new spoof requests.
- spoof:** A spoof request comprised of the fields found in a node report. The request must contain the vehicle position. All other fields are optional, relying on default values if a field is not specified in the spoof request. If the **name** is not specified, an auto-generated name will be used, beginning with "C" followed by an index incremented with each spoof request. Note: a spoof request may also be made through an incoming posting to the MOOS variable **SPOOF**.

An Example MOOS Configuration Block

An example MOOS configuration block may be obtained from the command line with the following:

```
$ pSpoofNode --example or -e
```

Listing 2.2: Example configuration of the pSpoofNode application.

```

1  =====
2  pSpoofNode Example MOOS Configuration
3  =====
4
5  ProcessConfig = pSpoofNode
6  {
7      AppTick    = 4
8      CommsTick  = 4
9
10     refresh_interval = 1          // Secs, default = 1
11     default_length = 5            // Meters, default = 5
12     default_vtype =               // Shape, default = kayak
13     default_group =               // Default = ""
14     default_vsource =             // Default = ""
15     default_color =               // Default = purple
16
17     default_hdg = 45              // Degrees, default=45
18     default_spd = 2               // M/sec, default=0
19     default_duration = 0          // Sec, default=0, never expire
20 }
```

3 Publications and Subscriptions for pSpoofNode

The interface for **pSpoofNode**, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ pSpoofNode --interface or -i
```

3.1 Variables Published by pSpoofNode

- **NODE_REPORT:** A node report representing the spoofed node.

3.2 Variables Subscribed for by pSpooofNode

The `pSpooofNode` application subscribes to the following MOOS variables:

- `APPCAST_REQ`: A request to generate and post a new apppcast report, with reporting criteria, and expiration.
- `SPOOF`: A request for generating a spoofed node.
- `SPOOF_CANCEL`: A request to cancel an ongoing spoofed node, based on vehicle name or group name.

Example `SPOOF` requests:

```
SPOOF = x=12,y=9,hdg=180,spd=0.1,name=zed,group=blue_team,type=kayak,
        vsource=ais,len=5,color=green,dur=20
SPOOF = x=12,y=9,hdg=180,spd=0.4
```

Example `SPOOF_CANCEL` requests:

```
SPOOF_CANCEL = vname=zed
SPOOF_CANCEL = group=redteam
```

3.3 Command Line Usage of pSpooofNode

The `pSpooofNode` application is typically launched with `pAntler`, along with a group of other modules. However, it may be launched separately from the command line. The command line options may be shown by typing:

```
$ pSpooofNode --help or -h
```

Listing 3.3: Command line usage for the pSpooofNode tool.

```
1 Usage: pSpooofNode file.moos [OPTIONS]
2
3 Options:
4   --alias=<ProcessName>
5       Launch pSpooofNode with the given process
6       name rather than pSpooofNode
7   --example, -e
8       Display example MOOS configuration block
9   --help, -h
10      Display this help message.
14   --interface, -i
15       Display MOOS publications and subscriptions.
16   --version, -v
17       Display the release version of pSpooofNode.
18   --web, -w
19       Open browser to: https://oceanai.mit.edu/ivpman/apps/pSpooofNode
20
21 Note: If argv[2] is not of one of the above formats
22       this will be interpreted as a run alias. This
23       is to support pAntler launching conventions.
```

4 Terminal and AppCast Output

Some useful information is published by `pSpooofNode` to the terminal on every iteration. An example is shown in Listing 4 below. This application is also appcast enabled, meaning its reports are published to the MOOSDB and viewable from any uMAC application or `pMarineViewer`. The counter on the end of line 2 is incremented on each iteration of `pSpooofNode`, and also serves as a heartbeat indicator. The "0/0" also on line 2 indicates there are no configuration or run warnings detected.

Listing 4.4: Example terminal or appcast output for `pSpooofNode`.

```
1 =====
2 pSpooofNode shoreside                                0/0(228)
3 =====
4 Config:
5   Default Type:    kayak
6   Default Color:   dodger_blue
7   Default Group:
8   Default VSource:
9   refresh Interv:  1
10
11 Postings: 36
12
13 VName  XPos  YPos   Hdg  Spd  Age
14 -----
15 ben    36.7 -107.9  0    0    12.1
```

Lines 4-7 of the output show the user configuration. Lines 13-15 show the most recent spoof request per vehicle. The requested position, heading and speed are shown. The Age column represents how long ago the spoof request was generated.